Aula 5



DEADLOCK

Ocorre quando dois ou mais processos ficam bloqueados permanentemente porque cada um está segurando um recurso que o outro precisa para continuar. Nenhum deles avança e nenhum libera o recurso retido.

- **Analogia:** Dois carros se encontram em uma ponte estreita de mão única, frente a frente. Nenhum dos motoristas aceita dar ré; logo, o trânsito trava por completo.
- **Estado do processo:** Bloqueado/parado (não consome CPU).
- **Quatro condições necessárias (Coffman):**
    1. *Exclusão Mútua:* O recurso só pode ser usado por um processo por vez.
    2. *Posse e Espera:* Um processo retém um recurso enquanto espera por outro.
    3. *Não Preempção:* Recursos não podem ser tomados à força.
    4. *Espera Circular:* Uma cadeia fechada de processos onde P_1 espera P_2, P_2 espera P_3, ..., e P_n espera P_1.

Estratégias de Tratamento de Deadlock:

Existem quatro abordagens formais:

1. **Avestruz (Ignorar):** O SO assume que o evento é raro e não faz nada (adotado pelo Linux e Windows em nível de usuário; se travar, o usuário encerra o processo).
2. **Prevenção (*Prevention*):** Eliminar a possibilidade quebrando matematicamente pelo menos uma das 4 condições de Coffman (ex.: exigir ordenação global para alocação de recursos para impedir espera circular).
3. **Evitação (*Avoidance*):** O SO analisa dinamicamente cada pedido de recurso para manter o sistema em um **Estado Seguro** (*Safe State*). O exemplo clássico que cai em prova é o **Algoritmo do Banqueiro** (Dijkstra).
4. **Detecção e Recuperação:** Deixa acontecer, roda periodicamente um algoritmo de busca de ciclo (usando Grafo de Alocação de Recursos) e recupera matando processos (*kill*) ou desfazendo ações (*rollback*).

INANIÇÃO/STARVATION

Ocorre quando um processo pronto para ser executado é perpetuamente ignorado pelo escalonador ou impedido de obter recursos porque outros processos (geralmente com maior prioridade) continuam recebendo esses recursos antes dele.

- **Analogia:** Uma pessoa esperando na fila de um pronto-socorro que nunca é chamada, pois novos pacientes chegam continuamente e passam à frente por terem triagem com prioridade mais alta.
- **Estado do processo:** Pronto para executar ou em espera, mas nunca selecionado.
- **Diferença para Deadlock:** O sistema continua progredindo como um todo; apenas uma ou mais threads específicas ficam sem atendimento.

- **Solução comum:** Algoritmos de *Aging* (envelhecimento), onde a prioridade de um processo aumenta progressivamente quanto mais tempo ele passa esperando.
- **Algoritmos propensos:** Políticas puras de escalonamento como **SJF (Shortest Job First)**, **SRTF** e filas de **Prioridade Estrita** são as maiores causadoras de starvation para processos longos ou de baixa prioridade.
- **Inversão de Prioridade (*Priority Inversion*):** Cenário onde um processo de alta prioridade fica em inanição esperando por um recurso detido por um processo de baixa prioridade, enquanto processos de média prioridade impedem o de baixa de rodar e liberar o recurso.

LIVELOCK

Ocorre quando dois ou mais processos mudam ativamente de estado em resposta às ações uns dos outros, mas essas mudanças não produzem nenhum progresso prático na execução.

- **Analogia:** Duas pessoas tentando passar por um corredor estreito. As duas dão um passo para a esquerda ao mesmo tempo para abrir passagem, depois ambas dão um passo para a direita simultaneamente, repetindo o desvio indefinidamente sem que nenhuma consiga passar.
- **Estado do processo:** Ativo/executando (consome ciclos de CPU em repetições inúteis).
- **Causa comum:** Algoritmos de recuperação de conflitos ou deadlocks excessivamente simétricos ou sem mecanismos de desempate aleatório (*backoff*).